

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Sistemas Operativos I

Trabajo Práctico Final

Integrantes:

Lautaro Capezio
Luciano Duarte

Índice

1. Introduccion	2
2. Arquitectura general	2
3. Diseño de concurrencia	2
3.1. Modelo adoptado	2
3.2. Justificacion de las decisiones clave	3
3.3. Exclusion mutua	3
3.4. Sincronizacion interna	4
3.5. Deadlocks y bloqueos	4
4. Robustez	5
4.1. Errores manejados	5
4.2. Estrategias de recuperacion	6
5. Comparacion de paradigmas	6
5.1. Que simplifico Erlang	6
5.2. Que complejizo Erlang	7
5.3. Ventajas de C para este problema	7
5.4. Desventajas de C para este problema	7
6. Conclusion	9

1. Introduccion

El trabajo implementa un sistema P2P descentralizado para una red local en el que cada nodo cumple simultaneamente el rol de servidor y cliente. El sistema utiliza UDP para descubrimiento y consenso de identificadores unicos, y TCP para busqueda, verificacion de integridad y transferencia de archivos. Ademas, soporta descargas paralelas por chunks desde multiples fuentes.

La implementacion se realizo en Erlang, por lo que el diseño de concurrencia se apoya en procesos livianos y paso de mensajes. Esta decision afecta de forma directa la estructura del programa, la forma de encapsular el estado, la estrategia de sincronizacion y la manera de tolerar fallas parciales.

2. Arquitectura general

El nodo se organiza alrededor de un proceso coordinador principal, implementado en `p2p_node.erl`, que inicia y detiene el resto de los componentes. A partir de ese coordinador se levantan procesos especializados:

- `file_manager`: mantiene la vista local de la carpeta compartida y atiende consultas sobre archivos.
- `node_registry`: conserva el conjunto de nodos conocidos y depura entradas inactivas.
- `discovery`: escucha mensajes UDP, participa del consenso de identificadores y registra nuevos nodos.
- `hello_broadcast`: envia periodicamente mensajes HELLO para anunciar presencia.
- `tcp_server`: acepta conexiones TCP y crea un proceso independiente por cliente.
- `download_manager`: coordina el estado de una descarga multi-fuente y asigna chunks a workers.
- `cli`: ejecuta la interfaz interactiva sin bloquear al coordinador.

La descomposicion por procesos cumple un doble objetivo. Por un lado, separa responsabilidades funcionales; por otro, evita compartir memoria mutable entre modulos concurrentes. Cada componente posee su propio loop de recepcion y su propio estado local.

3. Diseño de concurrencia

3.1. Modelo adoptado

El modelo de concurrencia de Erlang se basa en procesos livianos aislados, cada uno con heap propio y comunicacion mediante envio asincrono de mensajes. En esta aplicacion ese enfoque aparece de manera explicita:

- El coordinador del nodo se crea con `spawn` y conserva referencias a procesos hijos en su diccionario de proceso.
- `node_registry` mantiene su estado como un mapa interno y lo modifica solo dentro de su loop.

- `download_manager` centraliza el estado de cada descarga multi-fuente y responde pedidos de asignacion, progreso y finalizacion.
- `tcp_server` acepta conexiones y crea un proceso por cliente con `spawn_link`, permitiendo atencion concurrente de multiples pedidos.
- Las busquedas distribuidas lanzan un proceso por nodo remoto para consultar en paralelo.
- Las descargas multi-fuente lanzan varios workers por nodo, de modo que diferentes chunks puedan descargarse simultaneamente.

Esta estrategia reemplaza un diseño clasico con hilos compartiendo estructuras globales. En vez de tener tablas compartidas protegidas por mutexes, el estado queda encapsulado en procesos propietarios.

3.2. Justificacion de las decisiones clave

La primera decision importante fue utilizar un proceso coordinador principal para el ciclo de vida del nodo. Esto simplifica el arranque y apagado ordenado, porque un unico proceso conoce los PIDs relevantes y puede cerrar CLI, servidor TCP, registro, descubrimiento y broadcaster de manera consistente.

La segunda decision clave fue modelar los componentes con estado como servidores manuales basados en `receive`. En particular, `node_registry` y `download_manager` funcionan como monitores serializadores: reciben mensajes, actualizan su estado y vuelven al loop. De esa forma se evita que dos tareas modifiquen simultaneamente la misma estructura.

La tercera decision relevante fue paralelizar solo el trabajo que no requiere estado compartido complejo. Por ejemplo, el servidor TCP crea un proceso por conexion porque cada cliente maneja su propio socket; la busqueda distribuida crea un proceso por nodo remoto porque cada consulta es independiente; la descarga multi-fuente crea workers concurrentes, pero las decisiones de asignacion de chunks quedan centralizadas en `download_manager`.

3.3. Exclusion mutua

En esta implementacion, la exclusion mutua no se logra con mutexes sino con **propiedad exclusiva del estado**. El patron es el siguiente:

- Un proceso es el unico autorizado a modificar cierta estructura.
- Los demas procesos no acceden directamente al estado, sino que envian pedidos.
- El proceso propietario atiende esos pedidos secuencialmente en su mailbox.

Este esquema aparece claramente en dos modulos:

- `node_registry`: el mapa de nodos conocidos solo se modifica dentro de `loop/1`. Cuando llega `{add_node, ...}`, el proceso actualiza `last_seen`, elimina entradas obsoletas con misma IP:puerto y conserva una vista consistente.
- `download_manager`: el estado de chunks (`pending`, `downloading`, `complete`) y las asignaciones activas solo cambian dentro de un unico loop. Asi se evita que dos workers reciban el mismo chunk a la vez.

Desde el punto de vista conceptual, el mailbox del proceso cumple el rol que en C cumpliría una sección crítica protegida por mutex. La diferencia es que en Erlang la serialización está integrada al modelo y no requiere bloqueos explícitos alrededor de estructuras compartidas.

3.4. Sincronización interna

La sincronización entre componentes se resuelve con mensajes y timeouts. Algunos ejemplos relevantes:

- **Arranque del nodo:** `p2p_node:start/1` espera el mensaje `{node_started, Pid}` del coordinador antes de quedar bloqueado hasta el apagado.
- **Consultas al registro:** funciones como `get_all_nodes/0` y `get_node/1` envían una solicitud y esperan una respuesta puntual del proceso `node_registry`.
- **Asignación de chunks:** cada worker llama a `download_manager:assign_chunk/2`; el coordinador de descargas responde con un chunk disponible o con `no_chunks`.
- **Busqueda distribuida:** el proceso invocante lanza consultas paralelas y luego consolida respuestas con `collect_results`.
- **Descarga multi-fuente:** el proceso principal espera eventos `worker_done` y `worker_error` hasta que todos los chunks estén completos o se alcance un timeout global.

En todos estos casos, la sincronización es cooperativa y basada en protocolos de mensajes simples. No hay variables de condición ni semáforos expuestos al programador.

3.5. Deadlocks y bloqueos

En una implementación equivalente en C con hilos, este problema sería especialmente sensible a deadlocks por adquisición de múltiples mutexes en distinto orden, olvido de liberar locks, o esperas condicionales mal coordinadas. En esta solución, ese tipo de deadlock se reduce drásticamente porque no hay locks explícitos ni memoria compartida mutable entre procesos.

Sin embargo, Erlang no elimina todos los riesgos de bloqueo. Los principales riesgos pasan a ser:

- **Esperas indefinidas por respuesta:** un proceso puede quedar esperando un mensaje que nunca llega si el emisor falla o no responde.
- **Timeouts de red:** una operación TCP o UDP puede quedar demorada por desconexión, nodo caído o indisponibilidad del socket.
- **Starvation lógica:** un chunk podría quedar sin completar si no existiera una política de reasignación tras un fallo.

Estos riesgos se mitigaron de varias maneras:

- La mayoría de las operaciones sincrónicas sobre procesos internos usan `after` con timeouts, por ejemplo en `node_registry`, `download_manager` y `search`.

- Las operaciones TCP usan timeouts configurables para conexion, lectura de cabecera y recepcion de chunks.
- Cuando un worker falla al bajar un chunk, el coordinador lo vuelve a marcar como `pending`, habilitando el reintento por otro worker o por la misma fuente.
- Los workers pueden dejar de insistir con una fuente cuando detectan errores terminales como `node_not_found`, `connection_refused` o `connection_timeout`.

Por lo tanto, el diseño evita deadlocks clasicos de exclusion mutua, pero aun requiere manejo explicito de bloqueos logicos y de fallas en la comunicacion.

4. Robustez

4.1. Errores manejados

La robustez del sistema descansa en una combinacion de deteccion temprana de errores, timeouts y recuperacion por reintento o degradacion controlada. Los casos cubiertos mas importantes son los siguientes:

- **Fallo al abrir sockets UDP:** durante la solicitud de ID, si `gen_udp:open` falla, el modulo de descubrimiento informa el error, espera un segundo y reintenta.
- **Colision de identificadores:** si otro nodo rechaza un ID propuesto mediante `INVALID_NAME`, se espera un tiempo aleatorio y se vuelve a solicitar un identificador.
- **Desconexion o inactividad de nodos:** `node_registry` elimina nodos cuyo `last_seen` supera el umbral de inactividad configurado.
- **Errores de aceptacion TCP:** `tcp_server` captura errores en `accept` y continua escuchando para no derribar el servicio.
- **Excepciones en handlers de clientes:** cada conexion se procesa con `handle_client_safe/1`, que captura excepciones, registra el error y cierra el socket.
- **Timeouts de lectura:** el servidor y los clientes TCP usan timeouts para evitar quedar bloqueados indefinidamente esperando datos.
- **Archivo inexistente:** tanto para descarga completa como para chunks, si el archivo o fragmento no existe se responde con el codigo correspondiente de `not found`.
- **Errores de integridad:** la descarga simple valida `sha256` del archivo completo; la descarga multi-fuente reensambla el archivo y compara el hash con el informado por alguna fuente.
- **Falla en descarga de chunk:** si una descarga parcial falla, el chunk vuelve a estado `pending` para que pueda ser reasignado.
- **Fuente no confiable o caida:** el worker puede abandonarla cuando el error indica que insistir no tiene sentido.

4.2. Estrategias de recuperacion

No todos los errores se tratan igual. La estrategia cambia segun si el error es transitorio, definitivo o si afecta unicamente a una sub-tarea concurrente:

- **Reintento local:** se usa al abrir el socket UDP del consenso de ID.
- **Reasignacion de trabajo:** se usa en descargas por chunks; un fallo no invalida toda la descarga sino solo la unidad de trabajo afectada.
- **Aislamiento por proceso:** si falla el handler de una conexion TCP, no cae el servidor completo.
- **Degradacion controlada:** si no se puede obtener el checksum remoto, la descarga puede completarse igual, pero se informa que no se pudo verificar integridad remota.
- **Depuracion periodica:** el registro de nodos remueve entradas inactivas para evitar decisiones basadas en fuentes obsoletas.
- **Apagado ordenado:** el coordinador principal cierra procesos auxiliares para no dejar recursos o loops activos al salir.

5. Comparacion de paradigmas

5.1. Que simplifico Erlang

Para este tipo de aplicacion distribuida y concurrente, Erlang simplifico varias cuestiones estructurales:

- **Concurrencia mas barata:** lanzar un proceso por conexion, por consulta o por worker es natural y de bajo costo.
- **Aislamiento del estado:** al no compartir memoria, disminuye mucho el riesgo de corrupcion de estructuras y de carreras de datos.
- **Sincronizacion mas clara:** el modelo pedido-respuesta sobre mailboxes es facil de razonar y encaja bien con servicios como registro de nodos y gestor de descargas.
- **Tolerancia a fallas localizada:** una excepcion en un handler TCP afecta a esa conexion y no necesariamente al resto del sistema.
- **Comunicacion distribuida natural:** la semantica de procesos y mensajes se alinea muy bien con un sistema P2P, donde gran parte de la logica ya consiste en intercambio de mensajes.

En C, lograr el mismo nivel de paralelismo habria requerido threads o un esquema de `select/poll/epoll`, junto con una capa manual para coordinacion y proteccion de estructuras compartidas. Eso suele aumentar la complejidad accidental del codigo.

5.2. Que complejo Erlang

Tambien hubo costos y complejidades especificas del paradigma:

- **Mayor necesidad de diseñar protocolos:** al no haber acceso directo a memoria compartida, incluso consultas simples requieren definir mensajes, formatos y respuestas.
- **Sincronizacion explicita por mensajes:** cuando un proceso necesita una respuesta, hay que gestionar `receive`, correlacion de mensajes y timeouts.
- **Control mas manual fuera de OTP:** al implementar servidores con `spawn` y `receive` en lugar de `gen_server`, parte de la robustez queda en manos del desarrollador.
- **Depuracion de flujos concurrentes:** aunque se evitan data races, sigue siendo necesario rastrear interacciones entre procesos y orden de mensajes.

Es decir, Erlang simplifica la seguridad de concurrencia de bajo nivel, pero exige mayor disciplina en el diseño de protocolos entre actores.

5.3. Ventajas de C para este problema

Un enfoque en C habria tenido algunas ventajas puntuales:

- Mayor control fino sobre memoria, buffers y uso del sistema operativo.
- Posible mejor rendimiento bruto en ciertas rutas muy optimizadas de E/S o copiado de datos.
- Mayor cercania con primitivas POSIX tipicas de la materia, como threads, mutexes y variables de condicion.

Sin embargo, esas ventajas vienen acompañadas por una carga de implementacion significativamente mayor para una aplicacion de este tipo.

5.4. Desventajas de C para este problema

En un sistema P2P con descubrimiento, multiples conexiones simultaneas, workers de descarga y estado compartido, C habria agregado complejidades importantes:

- Necesidad de proteger estructuras globales como el registro de nodos o el estado de descarga mediante mutexes.
- Riesgo real de carreras de datos, deadlocks y errores por orden incorrecto de locking.
- Mayor dificultad para aislar fallas de una conexion individual sin comprometer threads o estructuras compartidas.
- Mayor trabajo manual para manejo seguro de memoria, buffers y limpieza de recursos ante errores intermedios.

Para una aplicacion distribuida con fuerte componente concurrente, estas desventajas son especialmente relevantes.

Para este trabajo, el paradigma de Erlang fue particularmente adecuado porque el problema esta dominado por coordinacion entre actores concurrentes y comunicacion en red, no por calculo intensivo local. En consecuencia, el modelo de procesos y mensajes simplifico la arquitectura general y redujo errores tipicos de concurrencia compartida.

6. Conclusion

El sistema implementado muestra una correspondencia fuerte entre el dominio del problema y el paradigma elegido. Al tratarse de un nodo P2P con descubrimiento, consultas remotas, transferencia de archivos y descargas paralelas, el modelo de Erlang permitio estructurar la solucion como un conjunto de procesos cooperativos, cada uno responsable de un fragmento bien definido del estado.

La exclusion mutua se resolvio sin mutexes mediante propiedad exclusiva del estado y serializacion por mailbox. La sincronizacion se realizo mediante mensajes y timeouts. Los deadlocks clasicos del modelo de hilos se evitaron, aunque siguio siendo necesario tratar bloqueos logicos y fallas de comunicacion. En robustez, el sistema incorpora reintentos, deteccion de nodos inactivos, aislamiento de errores por conexion, validacion de integridad y reasignacion de chunks fallidos.

En comparacion con una solucion en C, Erlang reduce la complejidad accidental de la concurrencia y mejora la claridad del diseño distribuido, a costa de requerir mas trabajo en la definicion de protocolos de mensajes y, en esta version, de no aprovechar completamente herramientas OTP como supervisores y comportamientos estandar. Aun asi, para la naturaleza de esta aplicacion, la eleccion de Erlang resulto ventajosa y coherente con los objetivos del trabajo.